INVERTIS UNIVERSITY, BAREILLY

B.Tech CSE – Third Year

APPLIED DATA STRUCTURES & ALGORITHMS LAB (BCS553)

LAB 6

Assignment 6: Binary Search on a Sorted Array

Assignment aim: Search a sorted array by repeatedly halving the search range.

Experiment No. 6.1 — Binary search for a present key

1. Experiment Title: Binary search for a present key

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Binary search works on sorted data and halves the candidate range after each comparison.

5. Algorithm / Procedure:
Set low and high bounds; inspect the middle element; discard the half that cannot contain the key; repeat until found or bounds cross.
Then compile and execute the Java program.

6. Program:

```
public class Task_01_Binary_search_for_a_present_key{
    static int binarySearch(int[] a, int key) {
        int low = 0, high = a.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (a[mid] == key) return mid;
            if (a[mid] < key) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 40));
    }
}
```

7. Input: Fixed input values are declared in the program: sorted array {10, 20, 30, 40, 50} and the task's search key.

8. Output:
3

Time / Space Complexity: O(log n) time; O(1) auxiliary space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What condition must the array satisfy?
- How is mid calculated safely?
- Why does binary search discard half the range?

Experiment No. 6.2 — Binary search for an absent key

1. Experiment Title: Binary search for an absent key

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Binary search works on sorted data and halves the candidate range after each comparison.

5. Algorithm / Procedure:
Compare the key with the middle element; repeatedly narrow the sorted range; return -1 when no range remains.
Then compile and execute the Java program.

6. Program:

```
public class Task_02_Binary_search_for_an_absent_key{
    static int binarySearch(int[] a, int key) {
        int low = 0, high = a.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (a[mid] == key) return mid;
            if (a[mid] < key) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 35));
    }
}
```

7. Input: Fixed input values are declared in the program: sorted array {10, 20, 30, 40, 50} and the task's search key.

8. Output:
-1

Time / Space Complexity: O(log n) time; O(1) auxiliary space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- Why is 35 not found?
- What happens when low becomes greater than high?
- Can binary search be used directly on an unsorted array?

Experiment No. 6.3 — Binary search for the first element

1. Experiment Title: Binary search for the first element

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Binary search works on sorted data and halves the candidate range after each comparison.

5. Algorithm / Procedure:
Start with the full sorted-array range; compare the middle value to the key; narrow the range until key 10 is found.
Then compile and execute the Java program.

6. Program:

```
public class Task_03_Binary_search_for_the_first_element{
    static int binarySearch(int[] a, int key) {
        int low = 0, high = a.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (a[mid] == key) return mid;
            if (a[mid] < key) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 10));
    }
}
```

7. Input: Fixed input values are declared in the program: sorted array {10, 20, 30, 40, 50} and the task's search key.

8. Output:
0

Time / Space Complexity: O(log n) time; O(1) auxiliary space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What is the index of the first element?
- What are low and high?
- What is binary search's worst-case time complexity?